CYBERSECURITY TRACK • PHP & WEB MODULE

Assignment 12

Secure File Upload System (PHP)

Done by: Abdulhadi Al-Ahdal

1. Project Overview

Develop a PHP server script that handles uploaded files implementing a Defense-in-Depth strategy[cite: 8]. The software prevents Web Shell uploads and extension manipulation by inspecting size, strict whitelisted extensions, and actual inner file contents, while randomly renaming files to prevent Path Traversal vulnerabilities[cite: 8].

2. Development Steps

1 File Ingestion & Size Check

Receive uploaded files through `$_FILES['uploaded_file']` and verify maximum size constraints[cite: 8].

2 Extension Whitelisting

Validate file extensions against strict whitelists such as `['jpg', 'png']` [cite: 8].

3

4

MIME Type Inspection

Inspect actual file content using the Fileinfo library to ensure headers were not manipulated[cite: 8].

Cryptographic Renaming

Generate secure random names using `bin2hex(random_bytes(16))` and transfer files via `move_uploaded_file` [cite: 8].

3. Technical Features & Utilities

finfo_file Validation: Verify actual MIME types using `finfo_file()` instead of relying on browser-sent headers[cite: 8].

Extension Extraction: Use `pathinfo($filename, PATHINFO_EXTENSION)` to accurately parse file types[cite: 8].

4. Implementation Source Code

`upload_handler.php`

PHP 8.x

```php
<?php
header("Content-Type: application/json; charset=UTF-8");

$uploadDir = "uploads/";
if (!file_exists($uploadDir)) {
    mkdir($uploadDir, 0755, true);
}

$response = ["status" => "error", "message" => "No file uploaded."];
```

```php
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['uploaded_file'])) {
    $file = $_FILES['uploaded_file'];

    // 1. Check upload errors
    if ($file['error'] !== UPLOAD_ERR_OK) {
        $response["message"] = "Upload failed with error code: " . $file['error'];
        echo json_encode($response, JSON_PRETTY_PRINT);
        exit;
    }

    // 2. Size limitation check (e.g., max 2MB)
    if ($file['size'] > 2 * 1024 * 1024) {
        $response["message"] = "File size exceeds maximum allowed limit (2MB).";
        echo json_encode($response, JSON_PRETTY_PRINT);
        exit;
    }

    // 3. Extension Whitelist Check
    $allowedExtensions = ['jpg', 'jpeg', 'png', 'pdf'];
    $fileExtension = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));

    if (!in_array($fileExtension, $allowedExtensions)) {
        $response["message"] = "Invalid file extension. Allowed: " . implode(', ', $allowedExtensions);
        echo json_encode($response, JSON_PRETTY_PRINT);
        exit;
    }

    // 4. MIME Type Inspection via Fileinfo
    $finfo = finfo_open(FILEINFO_MIME_TYPE);
    $mimeType = finfo_file($finfo, $file['tmp_name']);
    finfo_close($finfo);
```

```
    $allowedMimeTypes = [
        'image/jpeg' => 'jpg',
        'image/png' => 'png',
        'application/pdf' => 'pdf'
    ];

    if (!array_key_exists($mimeType, $allowedMimeTypes)) {
        $response["message"] = "Security alert: File content does not match allowed MIME types.";
        echo json_encode($response, JSON_PRETTY_PRINT);
        exit;
    }

    // 5. Generate secure cryptographically randomized file name
    $secureFilename = bin2hex(random_bytes(16)) . '.' . $fileExtension;
    $destination = $uploadDir . $secureFilename;

    if (move_uploaded_file($file['tmp_name'], $destination)) {
        $response = [
            "status" => "success",
            "message" => "File uploaded and verified securely.",
            "stored_filename" => $secureFilename,
            "detected_mime" => $mimeType
        ];
    } else {
        $response["message"] = "Failed to move uploaded file to secure storage.";
    }
}
```

```
echo json_encode($response, JSON_PRETTY_PRINT);
?>
```

5. Acceptance Criteria

- ✓ **Malicious File Rejection**
  Successful rejection of malicious payloads or modified extensions (e.g., PHP disguised as JPG)[cite: 8].
- ✓ **Secure Storage Transfer**
  Safe generation of secure names and transfer of verified authorized files to dedicated storage[cite: 8].